Game Data Capture Spec

Version 1

1 Canonical Format

This document describes the standard Game Data format that Odyssey will use as a normalised canonical source of data for downstream processing.

Internally we use a parquet file for storage but can ingest from a standard CSV file.

The CSV needs to follow the conventions and column names described below.

The parquet and the CSV use identical column names and conventions.

1.1 World Coordinate System

Right-handed Z-up (`RFU`)

```
      Z (up)
      |
      |   Y (forward/north)
      |  /
      | /
      +————————> X (right/east)
```

- X = right / east
- Y = forward / north
- Z = up
- Units: game-world units (typically metres)
- Handedness: right-handed (`cross(X, Y) = Z`)

1.2 Camera-to-World (C2W) Matrix

We will use Camera-to-World matrix to store the camera information as this is generally the form that is most easily captured from game data. Downstream, this can be converted to whatever the the training pipeline requires eg Poses that are world-to-camera/RDF

The C2W matrix will be using row-major, is 4×4 homogeneous, and RUB coordinate systems (x=right, y=up, z=backward)

$$\begin{bmatrix} \text{c2w\_m00} & \text{c2w\_m01} & \text{c2w\_m02} & \text{c2w\_m03} \\ \text{c2w\_m10} & \text{c2w\_m11} & \text{c2w\_m12} & \text{c2w\_m13} \\ \text{c2w\_m20} & \text{c2w\_m21} & \text{c2w\_m22} & \text{c2w\_m23} \\ \text{c2w\_m30} & \text{c2w\_m31} & \text{c2w\_m32} & \text{c2w\_m33} \end{bmatrix} = \begin{bmatrix} R_x & U_x & B_x & T_x \\ R_y & U_y & B_y & T_y \\ R_i & U_i & B_i & T_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Column	Name	Meaning
0 (R)	Right	Camera right axis in world. Horizontal when no roll (R_i = 0).
1 (U)	Up	Camera up axis in the world. Points roughly toward +Z when the camera is upright.
2 (B)	Backward	Opposite of the look direction. The camera looks along −column 2.
3 (T)	Translation	Camera position in world coordinates.

Constraints:

- Bottom row: `(0, 0, 0, 1)`
- Rotation determinant: `+1` (right-handed)
- Row-major storage: `c2w_mRC` means row R, column C

1.3 Camera Intrinsics

The camera intrinsics will be represented by a perspective matrix. In the parquet and CSV, we just require fx, fy, cx, cy though we'll have an extra optional fields for other standard coefficients

Generally , the perspective matrix looks like this

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

There is also a **camera_model** field. This gives us the camera model that is used. Most game captures will use PINHOLE but the others are provided for future extension.

Model	Required Columns
`PINHOLE`	`camera_fx`, `camera_fy`, `camera_cx`, `camera_cy`
`OPENCV`	Above + `camera_radial_k1`, `k2`, `camera_tangential_p1`, `p2`
`OPENCV_FISHEYE`	Above + `camera_radial_k1` .. `camera_radial_k4`

Vendors can compute pinhole intrinsics from FOV + resolution:

```
f_y = height / (2 × tan(fov_vertical / 2))
f_x = f_y                                        [for square pixels]
c_x = width / 2
c_y = height / 2
```

1.4 Input Data

Input events are stored at the video frame rate (one row per frame, same as camera data).

All data that is captured at a higher frame rate than the video frame rate should be accumulated.

Column	Type	Format
`input_keys`	string	Pipe-delimited active keys. Empty string = none. E.g. `W|Shift|+`
`input_actions`	string	Semantic in-game actions (pipe-delimited) e.g. “Jump”, “Forward | Jump”, “Enter Vehicle”, “Shoot”...
`input_mouse_buttons`	string	Pipe-delimited active buttons. E.g. `Left|Right`
`input_mouse_dx`	float	Mouse movement delta X (unit not specified)
`input_mouse_dy`	float	Mouse movement delta X (unit not specified)

1.4.1 key_binding.json

An exhaustive mapping of literal `input_keys` that are pressed within the recorded session mapped to the `semantic_actions` the game receives. For example: `“space_bar” : “jump”`.

key_binding.json should be tuned to the session players’ individual game settings.

For example: If the player has adjusted their game setting’s default mapping from `“space_bar” : “jump”` to `“space_bar” : “crouch”` the session’s key_binding.json should reflect the mapping to crouch.

Illustrative example (not prescriptive of structure):

JSON

```
input_config = {
       "gameplay": {
              "shift": "sprint",
              "w": "move_forward",
              "e": "interact",
              "r": "reload",
              "escape": "open_menu",
       },
       "menu": {
       "enter": "confirm",
       "backspace": "cancel"
       }
}
```

1.5 All Columns

Sessions may have camera data, input data, or both. Camera columns are null for sessions without camera capture — our viewer and data tables hide them automatically.

Column	Type	Required	Description
`frame_id`	int	yes	Zero-based frame index
`timestamp_ms`	int	yes	Milliseconds from start of capture
`c2w_m00` .. `c2w_m33`	float	camera	C2W 4×4 matrix (see above). Null if no camera.
`camera_model`	string	camera	COLMAP camera model name. Empty if no camera.
`camera_fx`	float	camera	Focal length X in pixels (`fx = fy` for square pixels)
`camera_fy`	float	camera	Focal length Y in pixels
`camera_cx`	float	camera	Principal point X in pixels (`width / 2`)
`camera_cy`	float	camera	Principal point Y in pixels (`height / 2`)
`camera_radial_k1` .. `k6`	float	no	Radial distortion coefficients
`camera_tangential_p1`, `p2`	float	no	Tangential distortion coefficients
`input_keys`	string	no	Active keys (pipe-delimited)
`input_actions`	string	no	Semantic in-game actions (pipe-delimited) e.g. “Jump”, “Enter Vehicle”, “Shoot”...
`input_mouse_buttons`	string	no	Active mouse buttons (pipe-delimited)
`input_mouse_dx`	float	no	Mouse movement delta X (unit not specified)
`input_mouse_dy`	float	no	Mouse movement delta X (unit not specified)

Note on intrinsics: For games with square pixels (most), `fx = fy`. Derive from projection matrix: `fy = proj[1][1] × height / 2`, `fx = fy`. Do NOT derive `fx` independently from `proj[0][0]` — that encodes the viewport aspect, not a different focal length.

1.7 Self-Check

Vendors can verify their data is correct:

Check	Expected	What It Means
`det(R) = +1`	Exactly +1	Right-handed rotation
`c2w_m20 ≈ 0`	Near zero for level cameras	Right axis is horizontal (no roll)
`c2w_m21 > 0`	Positive for upright cameras	Up axis has a positive Z component
`c2w_m23`	Reasonable height value	Position Z is height above ground
`c2w_m30, m31, m32 = 0`	Exactly zero	Standard affine matrix
$R \times R^T = I$	Identity (within float precision)	Rotation is orthonormal

2 Vendor Input Format

Vendors should submit data in the canonical format above to their specific cloudflare r2 bucket using credentials Odyssey provides. For legacy vendors whose data doesn't match, a one-time preprocessor converts to canonical form.

2.1 Upload Path

```
<vendor>/<game>/<mm-dd-yy>(of upload)/<session-id>/
```

2.2 Session Structure (Vendor Submission)

```
.../<session-id>/
  session.json       # metadata sidecar (required)
  frames.csv         # per-frame camera + input data (required)
  video.mp4          # screen recording (required)
  key_binding.json   # mapping from literal input_key to semantic_action. Should have full
                         coverage of all input_keys used within session
```

2.3 15% Quality Assurance Session Structure (Vendor Submission)

A random 15% of every batch delivery must include:

1. `.rrd` file to visualize video frames alongside frame-synced metadata in https://rerun.io/
2. The script/code used to generate those `.rrd` files

```
.../<session-id>/
  ...                ...
  session.rrd        # the .rrd file you've created that visualizes all frame synced data
  rrd_creation.py    # the script used to create the provided session's .rrd file
```

4 Appendix: Coordinate Conversion Reference

For internal use when writing preprocessors for legacy vendor data.

4.1 3-Letter Axis Notation

Letter	Meaning
R / L	right / left
F / B	forward / backward
U / D	up / down

Common source systems:

- `RFU` — already correct (GTA V, Unreal Engine)

- `RUB` — OpenGL/Three.js native: swap Y↔Z
- `FLU` — ROS: remap X→Y, Y→−X
- `RDF` — OpenCV: remap Y→−Z, Z→Y

4.2 Converting Camera Columns to OpenGL Convention

- Compute **forward** from the source matrix (may require Euler decomposition)
- **right** = normalize(forward × world_up) — always horizontal
- **up** = normalize(right × forward) — gravity-aligned
- **backward** = −forward
- **C2W** = [right | up | backward | position]
- Verify `det(R) = +1`

> ⚠ **Warning:** Do not use similarity transforms ($P \times M \times P^{-1}$) for axis swaps with sign changes — they can flip pitch/roll signs. Remap each vector individually instead.

4.3 (GTA V) Conversion Notes

- **Affected data:** r2 bucket
- **Source world:** `RFU` (X=east, Y=north, Z=up) — same as canonical
- **Source C2W:** Euler-encoded R_Y(yaw) × R_X(pitch), M10 = 0 always
- Heading: `atan2(−M20, M00)`
  - Pitch: `atan2(−M12, M11)`
- **Source intrinsics:** `FOV_Deg` (vertical) + `FOV_Axis` column
- **Source inputs:** comma-separated keys, "NONE" for empty
- **Conversion:** decompose Euler → reconstruct via cross products → compute f_x/f_Y from FOV + resolution → pipe-delimit inputs

4.4 (RDR2) Conversion Notes

- **Affected data:** r2 bucket
- **Source world:** Z-up (position Z has smallest range)
- **Source C2W:** DirectX view matrix (row-vector, left-handed, det = −1) — **do not use directly**
- **Source Euler:** `rot_euler = [pitch, roll, yaw]` in degrees
- Yaw reference: +Y axis (north)
  - forward = (−sin(yaw)×cos(pitch), cos(yaw)×cos(pitch), sin(pitch))
- **Source intrinsics:** projection matrix with `proj[0][0]` and `proj[5]`
  - Use f_Y = `proj[5] × height/2`, $f_x = f_Y$
- **Source inputs:** event-based JSONL (keyboard.jsonl, mouse.jsonl) at variable rate
- **Camera data:** optional — some sessions (e.g. game-data bucket) lack camera.jsonl
- **Frame rate:** camera at ~120fps, video at 30fps — resample to video fps

- **Conversion:** Euler → cross products → OpenGL C2W. Resample inputs to frame rate. Pipe-delimit keys.

4.5 Cyberpunk Notes

- **Affected data:** r2 bucket
- ⚠️ ***Sync issue with ~109.9 hrs of Cyberpunk delivered on and prior to 04/06/26. (Wall time was fetched from the game clock not global clock).***

4.6 Hogwarts Legacy Notes

- **Affected bucket:** r2 bucket
- Hogwarts Legacy data needs to go through the §4.2 column-reconstruction process before delivery — decompose the UE5 actor-space matrix, re-express the axes in right-handed world space, and assemble [right | up | backward | position]

Misc

[DRAFTING] Game_Data_Capture_Spec_v2